Algorithmic Methods of Data Mining and Laboratory

Master's Degree in Data Science, Sapienza Università di Roma
Faculty of Information Engineering, Informatics and Statistics

Department of Computer Science

Author: Gianmaria Romano

Reference Bibliography

C. Aggarwal: *Data Mining: The Textbook*
R. Zafarani, M. A. Abbasi, and H. Liu: *Social Media Mining: An Introduction*
J. Leskovec, A. Rajaraman, and J. Ullman: *Mining of Massive Datasets*
C. D. Manning, P. Raghavan, and H. Schütze: *Introduction to Information Retrieval*
J. Janssens: *Data Science at the Command Line*

Academic Year 2026/2027

Disclaimer

This document contains notes for the **Algorithmic Methods of Data Mining and Laboratory** course, given by **Professors Anagnastopolous and Siciliano** as part of the Master's Degree in Data Science at **Sapienza Università di Roma** during the **first** semester of Academic Year **2026/2027**.

Note: These notes may not cover all details of each lecture, so it is highly suggested to consult the Professor's official resources and materials.

Important: These notes are free to share and use, but please remember to credit me as the author and do not try to hide/remove this page.

Contacts

For further information or requests, feel free to contact me privately or reach out through my repository: https://github.com/GianmariaRomano/Data-Science-Notes.

License

These documents are distributed under the GNU General Public License 3.0, a form of copyleft intended for use on a manual, textbook or other documents. Material licensed under the current version of the license can be used for any purpose, as long as the use meets certain conditions:

- All previous authors of the work must be attributed.
- All changes to the work must be logged.
- All derivative works must be licensed under the same license.
- The full text of the license, unmodified invariant sections as defined by the author if any, and any other added warranty disclaimers (such as a general disclaimer alerting readers that the document may not be accurate for example) and copyright notices from previous versions must be maintained.
- Technical measures such as DRM may not be used to control or obstruct distribution or editing of the document.

Contents

1 Introduction to data mining **4**
- 1.1 Finding a definition of data mining . . . 4
- 1.2 Understanding data types . . . 4
  - 1.2.1 Numeric data . . . 5
  - 1.2.2 Categorical data . . . 5
  - 1.2.3 Document data . . . 5
  - 1.2.4 Set data . . . 6
  - 1.2.5 Ordered data . . . 6
  - 1.2.6 Graph data . . . 6
- 1.3 Storing data . . . 6

2 Algorithms **7**
- 2.1 Defining algorithms . . . 7
- 2.2 Complexity analysis . . . 8
  - 2.2.1 The "gap problem" in asymptotic analysis . . . 9
- 2.3 Recursive algorithms . . . 10
- 2.4 Sorting algorithms . . . 11
  - 2.4.1 Mergesort . . . 11
  - 2.4.2 Quicksort . . . 12
- 2.5 **NP**-completeness . . . 12
  - 2.5.1 Solving **NP**-complete problems via heuristics . . . 13
  - 2.5.2 Solving **NP**-complete problems via approximation algorithms . . . 13

3 Data structures **14**
- 3.1 The purpose of data structures . . . 14
- 3.2 Lists . . . 14
- 3.3 Heaps . . . 15
- 3.4 Priority queues . . . 15

4 Data relationships **16**
- 4.1 Distance functions . . . 16
  - 4.1.1 The L^p distance family . . . 16
  - 4.1.2 The edit distance . . . 17
- 4.2 Similarity functions . . . 18
  - 4.2.1 The Jaccard index . . . 18

4.2.2 The cosine similarity 18

5 Text mining **19**
5.1 Handling text data 19
5.2 The search engine problem 19
5.2.1 The nearest neighbours formulation 19
5.2.2 The Boolean algorithm 21

Chapter 1

Introduction to data mining

1.1 Finding a definition of data mining

While no unique definition exists yet, it is possible to view data mining as the use of efficient techniques for the analysis of very large collections of data and the extraction of useful and possibly unexpected patterns in data in order to make decisions across various areas.

Example: Descriptive and predictive data mining

Descriptive data mining aims to identify patterns and relationships within data using unsupervised learning techniques that can be applied to recommendation systems, such as Netflix or Spotify's engines, to analyse patterns in user behaviour.
On the other hand, predictive data mining focuses on using past observations to predict a target variable through supervised learning techniques.
For instance, CERN's *A Large Ion Collided Experiment* analyses heavy-ion collision data to investigate the properties of quark-gluon plasma.

1.2 Understanding data types

Since data can come in different forms, it is necessary to organize it in order to extract meaningful information or patterns.
In particular, data can be seen as a collection of attributes that describe some objects and, depending on the organization paradigm, data can be classified into three main types:

- **Structured data:** Data is organized according to a well-defined schema that is enforced during storage.
- **Semi-structured data:** Data can be organized according to a structural schema, although this organization paradigm may not be enforced during storage.
- **Unstructured data:** Data does not follow a pre-defined structure, requiring a data cleaning procedure to store it as (semi-)structured data before use.

Example: Different data types

Examples of data organization can be seen on the InfoStud platform, which enforces strict organization through dedicated constraints on certain types of data, and medical records, which follow some structural patterns but might contain errors or anomalous data.
By contrast, social media platforms, such as Twitter or Reddit, provide clear examples of unstructured data that do not follow an organizational structure.

1.2.1 Numeric data

If each record has the same fixed set of numeric attribute, it can be seen as a point in a multi-dimensional space, meaning that the dataset can be treated as a matrix.

	#doors	Horsepower	weight (kg)	price	length (m)	Final speed (km/h)	0-100m (sec)
Car 1	3	120	1520	15,000	3.10	195	13.7
Car 2	4	210	1660	29,000	4.22	248	8.2
Car 3	5	158	2100	32,500	4.92	210	11.4

Figure 1.1: Example of numeric record data.

1.2.2 Categorical data

Categorical data consists of a collection of records with a fixed set of categorical variables.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	High	No
2	No	Married	Medium	No
3	No	Single	Low	No
4	Yes	Married	High	No
5	No	Divorced	Medium	Yes
6	No	Married	Low	No
7	Yes	Divorced	High	No
8	No	Single	Medium	Yes
9	No	Married	Medium	No
10	No	Single	Medium	Yes

Figure 1.2: Example of categorical record data.

1.2.3 Document data

Using the "bag-of-words" representation, a document can be represented using a text vector that indicates the occurrences of certain words of interest within the document.

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

Figure 1.3: Example of document data.

However, text vectors are often vulnerable to the "curse of dimensionality", which denotes all the performance problems that arise when dealing with high-dimensional data.
For this reason, document data are often pre-processed using dimensionality reduction techniques or principal component analysis in order to focus on just the relevant information carried by the data.

1.2.4 Set data

Set data can be seen as an efficient way of representing collections through binary attributes.

TID	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Figure 1.4: Example of set data.

Note that this type of data is often used to compress document data into sets of words.

1.2.5 Ordered data

Ordered data consists of ordered sequences of values that come in handy for genomics or for time-series forecasting.

1.2.6 Graph data

Graph data allows to summarise web network data using a (directed) graph $G(V, E)$ whose nodes represent pages or users and whose edges denote connections, such as links or interactions, between pages or users.

1.3 Storing data

Most of the time, data is stored using a cloud infrastructure that consists of clustered server racks that can store large amounts of data that might not fit on a normal computer.
From a low-level perspective, computers normally store information as bytes or words that can be accessed by referencing their memory address.
More specifically, whenever a program must be executed, the CPU receives a portion of the available memory and, after allocating space for the instructions and the stack frame, it can use the remaining space to allocate the variables that will be used during execution.

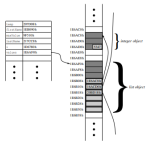


Figure 1.5: Memory management in Python.

Chapter 2

Algorithms

2.1 Defining algorithms

An algorithm can be seen as a set of instructions to solve a given problem.
In particular, an algorithm is evaluated based on its correctness, which determines whether the algorithm works on every possible input or not, and its efficiency, which measures the time taken by the algorithm to find a solution to the given problem.

Example: The maximum value of an array

Consider an algorithm that finds the maximum value in an array of n elements:

Data: An array A of size n
Result: $\max(A)$
$m \leftarrow A[1]$;
for $i \leftarrow 1$ **to** n **do**
  if $A[i] > m$ **then**
    $m \leftarrow A[i]$;
return m;

The correctness of this algorithm can be proved by induction on the number of steps.

- For the base case, given by $k = 1$, $A[k]$ is trivially the maximum value.
- Assuming the algorithm is correct on an input of size k, the goal is to prove its correctness on an input of size $(k + 1)$ as well.
  Claim that, after k steps, $m = \max_{i \in \{1,\dots,k\}} (A[i])$: it is possible to see that m will be updated if and only if $A[k + 1] > m$.

Additionally, looking at the pseudocode allows to bound the running time of this algorithm to at most $3n + 2$ moves on an input of size n.

Generally speaking, an algorithm is written using pseudocode in order to provide a language-independent idea of the solution to the given problem.

2.2 Complexity analysis

The running time of an algorithm is a function $T(n)$ that describes the number of steps done by the algorithm to solve a problem on an input of size n.
Since algorithm efficiency often depends on the code or machine that is used, it is often sufficient to find an estimate of the running time of an algorithm through asymptotic analysis.

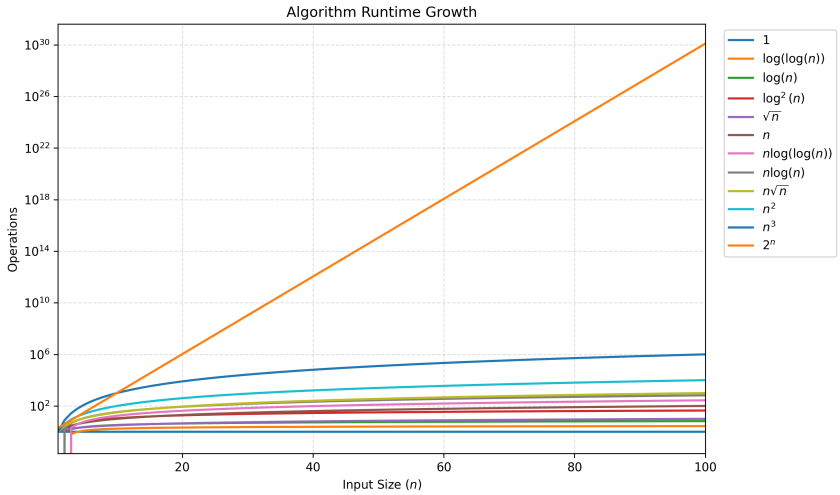


Figure 2.1: Visual comparison of common running time growth rates.

Therefore, given a function $g : \mathbb{N} \to \mathbb{R}^+$, it is possible to define the following asymptotic bounds:

- **Upper bound:** $T(n) = O(g(n))$ if it is possible to find $c > 0$ and $n_0 \geq 0$ such that

$$0 \leq T(n) \leq c \cdot g(n) \ \forall n \geq n_0$$

- **Lower bound:** $T(n) = \Omega(g(n))$ if it is possible to find $c > 0$ and $n_0 \geq 0$ such that

$$0 \leq c \cdot g(n) \leq T(n) \ \forall n \geq n_0$$

- **Tight bound:** $T(n) = \Theta(g(n))$ if it possible to find $c_1, c_2 > 0$ and $n \geq n_0$ such that

$$0 \leq c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n) \ \forall n \geq n_0$$

Example: The prefix sum problem

The following algorithm computes the prefix sum for an array of n elements:

Data: An array A of size n
Result: An array S of size n such that $S[i] = \sum_{k=1}^{i} A[k]$
$n \leftarrow len(A)$;
for $i \leftarrow 1$ **to** n **do**
  $S[i] \leftarrow 0$;
  for $j = 1$ **to** i **do**
    $S[i] \leftarrow S[i] + A[j]$;
return S;

Since this solution computes $i \leq n$ operations at each iteration, its running time can be bounded as $T(n) = n^2 + 3n + 2 = O(n^2)$.
However, since $T(n) = \Omega(n)$, there is a gap between the lower and upper running time bounds, requiring further investigation to find a better or tighter bound.
For this reason, one can opt for an alternative algorithm that re-uses a previously computed prefix $S[i-1]$ to compute $S[i]$ in constant time.

Data: An array A of size n
Result: An array S of size n such that $S[i] = \sum_{k=1}^{i} A[k]$
$n \leftarrow len(A)$;
$S[1] \leftarrow A[1]$;
for $i \leftarrow 2$ **to** n **do**
  $S[i] \leftarrow S[i-1] + A[i]$;
return S;

In fact, this strategy allows to reduce the overall time complexity to the optimal value of $T(n) = \Theta(n)$.

2.2.1 The "gap problem" in asymptotic analysis

The gap problem is a situation in which the lower and upper running time bounds of an algorithm do not match, meaning that no tight bound for the algorithm's complexity can be defined.

Example: Matrix multiplication

Suppose you want to compute the product between two matrices of size $n \times n$: the running time of the standard algorithm has a lower bound of $\Omega(n^2)$, which comes from the need to read and write each element in the product matrix, and an upper bound of $O(n^3)$, which denotes the overall computational cost.
Since the lower and upper bounds do not match, it is not possible to define a tight running time bound for the matrix multiplication algorithm.

2.3 Recursive algorithms

In some cases, it is possible to solve a problem using a recursive algorithm, which makes use of a function that (cyclically) calls itself to solve smaller pieces of a complex problem.

Example: The Fibonacci sequence

It is possible to compute a generic term of the Fibonacci sequence using a recursive algorithm.

Data: A non-negative integer n
Result: The nth element of the Fibonacci sequence
if $n \leq 1$ **then**
  return 1;
$f_1 \leftarrow Fibonacci(n-1)$;
$f_2 \leftarrow Fibonacci(n-2)$;
return $f_1 + f_2$;

In particular, the running time of this algorithm can be formulated as

$$T(n) = \begin{cases} 2 & \text{if } n = 0 \text{ or } n = 1 \\ T(n-1) + T(n-2) + 3 & \text{if } n \geq 2 \end{cases}$$

Unrolling this formulation reveals that

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 3 \geq T(n-2) + T(n-2) = 2T(n-2) \\ &\geq 2 \cdot 2T(n-4) = 4T(n-4) \geq \cdots \geq 2^i T(n-2i) \geq \dots \\ &\geq 2^{\frac{n}{2}} T\left(n - 2\left(\frac{n}{2}\right)\right) = 2^{\frac{n}{2}} T(0) = 2^{\frac{n}{2}+1} = O(2^{\frac{n}{2}}) \end{aligned}$$

Since the recursive solution runs in exponential time, it can be more convenient to opt for a trivial constant-time algorithm based on Binet's formula or for a linear-time iterative algorithm.

Data: A non-negative integer n
Result: The nth element of the Fibonacci sequence
if $n \leq 1$ **then**
  return 1;
previous $\leftarrow 0$;
current $\leftarrow 1$;
for $i \leftarrow 2$ **to** n **do**
  new $\leftarrow$ *previous* + *current*;
  previous $\leftarrow$ *current*;
  current $\leftarrow$ *new*;
return *current*;

2.4 Sorting algorithms

A sorting algorithm takes an input array and rearranges its elements in (ascending) order, typically using numerical or lexicographical criteria.
These algorithms come in handy to improve the efficiency of other algorithms that require or work well with sorted data.

2.4.1 Mergesort

Mergesort is a "divide-et-impera" algorithm that recursively splits an array into smaller sub-lists until they are trivially sorted before merging them back together.

Algorithm 1: The Mergesort algorithm.

Data: An array A
Result: A sorted permutation of A
$n \leftarrow len(A)$;
if $n \leq 1$ **then**
  return A;
$A_1 \leftarrow A[1, \ldots, \lceil \frac{n}{2} \rceil]$;
$A_2 \leftarrow A[\lceil \frac{n}{2} \rceil + 1, \ldots, n]$;
$A'_1 \leftarrow MergeSort(A_1)$;
$A'_2 \leftarrow MergeSort(A_2)$;
$A' \leftarrow merge(A'_1, A'_2)$;
return A';

Looking at the pseudocode, the running time of the algorithm can be formulated as

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + c_2 n + c_3 & \text{otherwise} \end{cases}$$

Unrolling the recursive case of this formulation reveals that

$$\begin{aligned} T(n) &= 2\left(2T\left(\frac{n}{4}\right) + c_2\frac{n}{2} + c_3\right) + c_2 n + c_3 \\ &= 4T\left(\frac{n}{4}\right) + 2c_2 n + 3c_3 \\ &= \ldots \\ &= 2^i T\left(\frac{n}{2^i}\right) + i c_2 n + (2^i - 1)c_3 \end{aligned}$$

Since the recursive calls can be executed until the base case, where $\frac{n}{2^i} = 1 \Leftrightarrow i = \log_2(n)$, the recurrence can be solved as

$$\begin{aligned} T(n) &= 2^{\log_2(n)} T(1) + c_2 n \log_2(n) + (2^{\log_2(n)} - 1)c_3 \\ &= cn + c_2 n \log_2(n) + (n-1)c_3 \\ &= c_2 n \log_2(n) + (c + c_3)n - c_3 = \Theta(n \log n) \end{aligned}$$

2.4.2 Quicksort

Quicksort is a randomized "divide-et-impera" algorithm that randomly selects a pivot and partitions the array into two sub-list, one containing the elements smaller than the pivot and the other containing the elements greater than the pivot, that are recursively sorted using the same strategy.

Algorithm 2: The Quicksort algorithm.

Data: An array A
Result: A sorted permutation of A
$n \leftarrow len(A)$;
if $n \leq 1$ **then**
  return A;
Pick a random $p \in \{1, \dots, n\}$;
Compare each element in A with $A[p]$, except for the pivot itself;
$A_1 \leftarrow [elements \leq A[p]]$;
$A_2 \leftarrow [elements > A[p]]$;
$A'_1 \leftarrow QuickSort(A_1)$;
$A'_2 \leftarrow QuickSort(A_2)$;
return $A'_1 + [A[p]] + A'_2$;

In this setting, the worst-case scenario happens if the algorithm consistently splits the array into maximally unbalanced sub-arrays, meaning that the chosen pivot is either the first or the last element in the array, causing the running time to degrade to $O(n^2)$.
However, since the pivot is always chosen uniformly at random, the worst-case scenario tends to be extremely unlikely as the expected running time of this algorithm can be bounded by $\mathbb{E}(T(n)) = O(n \log(n))$.

2.5 NP-completeness

When studying algorithms and computational complexity, (decision) problems can be classified depending on how hard they are to solve.
For this reason, it is possible to define the complexity classes **P**, which includes all decision problems that can be solved in polynomial time using a deterministic algorithm, and **NP**, which includes all decision problems for which a proposed solution can be verified in polynomial time.
While it is trivially known that $\mathbf{P} \subseteq \mathbf{NP}$, the question on whether $\mathbf{P} = \mathbf{NP}$ remains open.
In this setting, a problem **X** is said to be "**NP**-complete" if $\mathbf{X} \in \mathbf{NP}$ and **X** is **NP**-hard, which means that every problem $\mathbf{Y} \in \mathbf{NP}$ can be reduced to **X** in polynomial time.

Example: The travelling salesman problem

When formulated as a decision problem, the travelling salesman problem aims to determine whether a graph $G = (V, E)$ contains a path that visits all node in the graph while maintaining an overall cost of at most c.
Using some polynomial time reductions, it is possible to prove that this problem is **NP**-complete.

To prove that a problem **X** is **NP**-complete, it is typically sufficient to show that $\mathbf{X} \in \mathbf{NP}$ and that

some problem $\mathbf{Y} \in \mathbf{NP}$ can be reduced in polynomial time to $\mathbf{X}$.

2.5.1 Solving NP-complete problems via heuristics

Based on "metaheuristics", which represent high-level algorithmic frameworks for problem solving, heuristics provide a problem-specific rule to solve a **NP**-complete problem.
While heuristics are easy to implement and can provide acceptable solutions in reasonable time, they do not guarantee optimality and might struggle to generalize on different instances of the problem.

2.5.2 Solving NP-complete problems via approximation algorithms

In the context of optimization problems, a c-approximation algorithm is an algorithm that is guaranteed to return a solution within a factor c of the optimal solution in polynomial time.

Example: Approximating the travelling salesman problem

While the travelling salesman problem cannot be solved in polynomial time, there are some approximation algorithms that, for a cost function that satisfies the triangle inequality, are guaranteed to find a solution whose value is at most two times the optimal value.

Chapter 3

Data structures

3.1 The purpose of data structures

A data structure is a way of storing and organizing data to optimize or simplify certain tasks.
A data structure is first defined as an "abstract data type", which describes the operations supported by the structure, and then implemented as a "concrete data structure", which describes how data is stored and the way certain operations are performed.

3.2 Lists

A list is an abstract data type that is used to store a sequence of objects, each indicated by an index, while supporting the following operations:

- `L.insert(x, i)`: The value `x` gets inserted at position `i`.
- `L.remove(i)`: The value located in position `i` gets removed.
- `L.get(i)`: The list returns the value located in position `i`.
- `L.set(i)`: The value located in position `i` is set to `x`.

A list can be implemented using an array, which is a contiguous data structure where elements are stored sequentially and accessed using a positional index, or using a linked list, which is a non-contiguous data structure where elements can be stored in different memory locations and accessed using pointers.

Data Structure	**Insertion**	**Removal**	**Retrieval**	**Setting**
Array	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Linked List	$\Theta(1)$	$\Theta(1)$ or $\Theta(n)$	$\Theta(1)$ or $\Theta(n)$	$\Theta(1)$ or $\Theta(n)$

Table 3.1: Comparing the complexity of list operations for different data structures.

Overall, arrays are simple to manage but more computationally efficient, whereas linked lists are flexible and efficient but problematic to manage.

3.3 Heaps

A heap is a concrete data structure that stores data using a (binary) tree that satisfies the following structural properties:

- The tree is build top-down, starting from the root and and filling each tree level left-to-right before moving to the next one.
  This implies that the height of a tree containing n nodes will be $\lceil \log_2(n) \rceil$.
- The value of a node is always greater or equal than the values of its children, which, however, are not necessarily sorted because this property only holds locally.
- If a heap is implemented as an array, the child nodes of the element at position i are located at positions $2i$ and $(2i+1)$, respectively, whereas the parent node is located at position $\left\lfloor \frac{i}{2} \right\rfloor$.

For this reason, a heap is useful to keep track of the maximum or minimum value in an array.

3.4 Priority queues

A priority queue is an abstract data structure that stores a set of objects, each with an associated priority, while supporting the following operations:

- `Q.insert(x)`: The value `x` is inserted at the back of the queue.
- `Q.peekMax()`: The queue returns the maximum value.
- `Q.removeMax()`: The queue removes and returns the maximum value.

While a priority queue can be implemented using an array or a (doubly) linked lists, these data structures tend to be quite inefficient when peeking or removing the maximum value.
For this reason, priority queues are often implemented using heaps because the heap property allows to make queue operations more efficient.

Data Structure	**Insertion**	**Peeking**	**Removal**
Linked List/Array	$\Theta(1)$ or $\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Sorted Array	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Heap	$\Theta(\log(n))$	$\Theta(1)$	$\Theta(\log(n))$

Table 3.2: Comparing the complexity of queue operations for different data structures

Chapter 4

Data relationships

4.1 Distance functions

Since most data mining problems focus on describing relationships between objects within a "universe" $\mathcal{U}$, object discrepancies are measured using a distance function $d : \mathcal{U} \times \mathcal{U} \to \mathbb{R}^+$ such that

1. The function is non-negative, meaning that $d(x, y) \geq 0 \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$, with $d(x, y) = 0$ if and only if $x = y$.
2. The function is symmetric, meaning that $d(x, y) = d(y, x) \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$.
3. The function obeys the triangle inequality, meaning that $d(x, y) \leq d(x, z) + d(z, y) \ \forall x, y, z \in \mathcal{U}$.

4.1.1 The L^p distance family

If $\mathcal{U} = \mathbb{R}^m$, the distance between two m-dimensional vectors can be measured using the L^p distance function, which, for an arbitrary $p \in \mathbb{N}$, is defined as

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^{m} |\mathbf{x}_j - \mathbf{y}_j|^p \right)^{\frac{1}{p}} = \|\mathbf{x} - \mathbf{y}\|_p$$

As $p \to +\infty$, the L^p distance function becomes the Chebyshev distance, which is defined as

$$d_\infty(\mathbf{x}, \mathbf{y}) = \lim_{p \to +\infty} d_p(\mathbf{x}, \mathbf{y}) = \max_{1 \leq j \leq m} \{|\mathbf{x}_j - \mathbf{y}_j|\} = \|\mathbf{x} - \mathbf{y}\|_\infty$$

Example: Common distances between two vectors

Consider the vectors $\mathbf{x} = \begin{pmatrix} 3 & 7 & 0 & 2 & 1 \end{pmatrix}$ and $\mathbf{y} = \begin{pmatrix} 4 & 5 & 10 & 7 & 3 \end{pmatrix}$: using some common distances from the L^p family results in

$$\begin{aligned} d_1(\mathbf{x}, \mathbf{y}) &= 1 + 2 + 10 + 5 + 2 = 20 \\ d_2(\mathbf{x}, \mathbf{y}) &= 1 + 4 + 100 + 25 + 4 = \sqrt{134} \approx 11.57 \\ d_\infty(\mathbf{x}, \mathbf{y}) &= 10 \end{aligned}$$

Furthermore, choosing $p = 0$ causes the L^p distance to become the Hamming distance, which is defined as

$$d_0(\mathbf{x}, \mathbf{y}) = |\{j : \mathbf{x}_j \neq \mathbf{y}_j\}|$$

4.1.2 The edit distance

A string can be seen as a sequence whose sub-sequence is any sequence that contains a subset of the characters, not necessarily consecutive, of the string: for two strings S and T, a common sub-sequence is any sequence that is a sub-sequence for both S and T.
Using this information, it is possible to define the edit distance between two strings S and T as the minimum number of insertions, deletions and, in some cases, substitutions that are needed to transform S into T, resulting in

$ED(S, T) = |S| + |T| - 2LCS$, where LCS is the length of the longest common sub-sequence.

Example: Computing the edit distance between two strings

Assuming no substitution can be performed, compute the edit distance between the strings $S = CATGA$ and $T = ACTGC$:

$$S = CATGA \overset{del}{\to} ATGA \overset{ins}{\to} ACTGA \overset{ins}{\to} ACTGAC \overset{del}{\to} ACTGC = T$$

Since transforming S into T required four moves, it will be the case that $ED(S, T) = 4$.

While computing the edit distance between two strings typically requires exponential time, it is possible to cut complexity down to polynomial time using a dynamic programming algorithm.

Proof:

Given two strings S and T, of length n and m respectively, define S^i as the sub-sequence containing the first i characters of S and T^j as the sub-sequence containing the first j characters of T: it is possible to prove by induction that the edit distance between S^i and T^j can be recursively computed as

$$ED(S^i, T^j) = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ ED(S^{i-1}, T^{j-1}) & \text{if } i, j > 0 \text{ and } S[i] = T[j] \\ 1 + \min\{ED(S^{i-1}, T^j), ED(S^i, T^{j-1})\} & \text{if } i, j > 0 \text{ and } S[i] \neq T[j] \end{cases}$$

Knowing that $T(i, j) = c + T(i - 1, j) + T(i, j - 1)$, let $N = i + j$ and write

$$T(i, j) = T(N) = 2T(N - 1) + c \geq 2T(N - 1) \geq 4T(N - 2) = \cdots = 2^l T(N - l)$$

Since the recursive calls can go on until one of the strings becomes empty, which happens after $l \geq \min\{n, m\}$ steps, the recursive algorithm will require $T(n, m) = \Omega(2^{\min\{n,m\}})$ time. However, by looking at the recursive definition of $ED(S^i, T^j)$, one can implement the recursive call using a $(n + 1) \times (m + 1)$ table of optimal values, resulting in $O(nm)$ complexity.

It should be noticed that, despite a small change in the recursive formulation, this strategy can be applied even when the edit distance allows to perform substitutions as well.

4.2 Similarity functions

Object similarities can be described using a similarity function $s : \mathcal{U} \times \mathcal{U} \to \mathbb{R}^+$ such that

1. The function is non-negative, meaning that $s(x, y) \geq 0 \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$, and takes maximum value when $x = y$.
2. The function is symmetric, meaning that $s(x, y) = s(y, x) \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$.

Additionally, since similarity functions are often normalized, they can be turned into distance functions by setting $d(x, y) = 1 - s(x, y)$.

4.2.1 The Jaccard index

When working with set data, the Jaccard index can be used to measure the similarity between the elements of two sets A and B by computing

$$s(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

In particular, the Jaccard index can be expressed as a distance by setting

$$d(A, B) = 1 - s(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} = \frac{|A| + |B| - 2|A \cap B|}{|A \cup B|}$$

4.2.2 The cosine similarity

If $\mathcal{U} = \mathbb{R}^m$, the cosine similarity measures how similar the directions of two vectors are in terms of the angle between them.
Using the dot product, this metric can be formulated a

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos(\theta) \quad \Longrightarrow \quad \cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2} = \frac{\sum_{i=1}^{m} \mathbf{x}_i \mathbf{y}_i}{\sqrt{\sum_{j=1}^{m} \mathbf{x}_j^2} \sqrt{\sum_{j=1}^{m} \mathbf{y}_j^2}}$$

Therefore, this similarity will be maximum when the two vectors are parallel, meaning that they point towards the same direction, and minimum when the two vectors are anti-parallel, meaning that they point towards different directions.

Chapter 5

Text mining

5.1 Handling text data

Documents are typically processed using the "bag-of-words" model, which considers raw text as a vector that counts the occurrence of each word in the document.
However, before creating the vector, the extracted raw text first undergoes a pre-processing pipeline that normally consists of the following steps:

1. **Tokenization:** The document is split into a sequence of words or tokens.
2. **Stopword removal:** Common words that provide little semantic value are removed from the document in order to reduce computational costs and noise.
3. **Normalization:** Text features are standardized in order to avoid syntax-related or case-sensitive mismatches.
4. **Stemming:** Words are truncated to their root in order to group similar concepts together.

For this reason, text mining is often performed using the tools provided by the `nltk` library.

5.2 The search engine problem

Given a corpus of n pre-processed documents $\mathcal{C} = \{D^i\}_{i=1}^n$ and a query $\mathcal{Q}$, a search engine aims to find the k documents in the corpus that are closest to the query.
This problem can be interpreted as a "nearest neighbours" problem, which measures relevance in terms of geometric proximity, or as a "Boolean algorithm", which evaluates queries using dedicated data structures.

5.2.1 The nearest neighbours formulation

Using the "vector space" model, which treats each document $D^i \in \mathcal{C}$ as a text vector $\mathbf{d}^i \in \mathbb{R}^m$ within a vocabulary of terms, the search engine problem can be solved as a nearest neighbours problem.
However, checking document relevance using the Euclidean distance tends to yield misleading results because the algorithm ignores document sizes and term relevance.

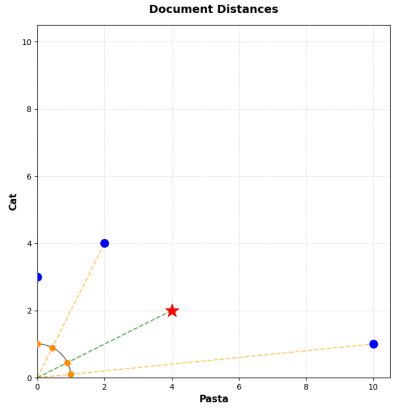


Figure 5.1: The search engine problem as a nearest neighbours problem in $\mathbb{R}^2$.

Concerning the first issue, one can prioritize vector directions and model proximity between two documents $D^i, D^j \in \mathcal{C}$ by measuring the cosine similarity between their text vectors, meaning that

$$s(D^i, D^j) = \cos(\theta) = \frac{\mathbf{d}^i \cdot \mathbf{d}^j}{\|\mathbf{d}^i\|_2 \|\mathbf{d}^j\|_2}$$

The second problem can instead be addressed using the "TF-IDF" representation: rather than viewing each text vector as a collection of raw frequencies TF^i_j, which simply measure the occurrence of term j in document D^i, it is possible to introduce a sequence of non-negative weights $w_1, \dots, w_m$ such that a text vector can be formulated as

$$\mathbf{d}^i_j = TF^i_j w_j = TF^i_j \log\left(\frac{n}{n_j}\right), \text{ where } n_j = |\{i : \mathbf{d}^i_j > 0\}|.$$

Knowing that a query $\mathcal{Q}$ can be treated as a text vector $\mathbf{q}$, it is possible to formulate the nearest neighbours problem using the following algorithm:

Algorithm 3: The search engine problem as a nearest neighbours problem.

Data: A corpus $\mathcal{C} = \{\mathbf{d}^i\}_{i=1}^n$, a query vector $\mathbf{q}$ and an integer $k \in \mathbb{N}$.
Result: The k closest documents to $\mathbf{q}$.
$scores \leftarrow [0] * n$;
for $i \leftarrow 1$ **to** n **do**
  $scores[i] \leftarrow \cos(\mathbf{q}, \mathbf{d}^i)$;
$scores \leftarrow sort(scores)$;
return *the indices of the* k *closest documents*;

While this approach is correct and easy to implement, its running time of $O(nm)$ makes it very inefficient for real-life search engines.

5.2.2 The Boolean algorithm

An alternative formulation of the search engine problem consists of finding the documents in the corpus that contain all the terms in the query vector.

Algorithm 4: The search engine problem as a Boolean algorithm.

Data: A corpus $\mathcal{C} = \{\mathbf{d}^i\}_{i=1}^n$, a query vector $\mathbf{q}$ and an integer $k \in \mathbb{N}$.
Result: All the documents that contain each term in $\mathbf{q}$.
$results \leftarrow \{\}$;
for $i \leftarrow 1$ **to** n **do**
  if $\mathbf{d}^i$ *contains all terms of* $\mathbf{q}$ **then**
    $results \leftarrow results \cup \{i\}$;
return *results*;

While this approach has a similar complexity to the nearest neighbours formulation, its efficiency can be improved by introducing an "inverted index" data structure that, for each term in the vocabulary, stores a posting list containing all the documents that contain that term.

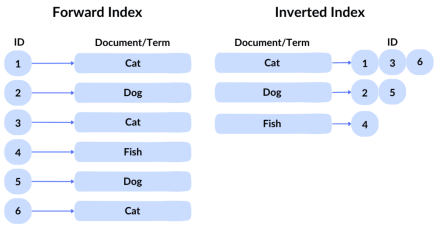


Figure 5.2: The "inverted index" data structure.

In fact, using inverted indexing allows to reduce the complexity of a query to the order of the sum of the lengths of the matching posting lists.
In particular, since this data structure can handle any Boolean query, it can be used to optimize the nearest neighbours formulation as well, focusing exclusively on the documents that share at least one term with the query and for which $\cos(\mathbf{q}, \mathbf{d}^i) > 0$.
Therefore, for each term in the query, the posting list now stores both the matching documents and the corresponding weighted frequency scores so that, since the length of the query does not influence the final scores, it is sufficient to compute

$$\cos(\theta) \propto \frac{\mathbf{q} \cdot \mathbf{d}^i}{\|\mathbf{d}^i\|_2} = \frac{\sum_{j=1}^m \mathbf{q}_j \cdot (TF_j^i w_j)}{\sqrt{\sum_{j=1}^m TF_j^i w_j}}, \text{ where } \mathbf{d}^i \text{ can be stored in a dedicated file.}$$